SRAF　專案進度提報

從人工提問 SRAF，走向 Excel 自動排程產情

工作成果｜目前開發進度｜下一階段｜導入效益

主管簡報

盤點日期： 2026 / 10 / 12

資料依據： Hosted GitHub hun186/poc-agent 、專案設計與驗收紀錄

這幾週做的：將「人工提問 SRAF」轉為「Excel 排程自動產情」

過去由人閱讀 Excel、透過 OpenWebUI 提問；現在補上要項匯入、排程控制與產情報告交付的產品能力。

10 項

PR #277–#286 已合併

Product 設計與後端階段成果

P1.7

目前進行中

輸入替換與版本衝突控制

2 種

任務執行模式

單次執行／週期性排程

1 條

共用執行主幹

延用 Import → Schedule → Agent → DOCX

現在的準確定位

已完成：後端邏輯與重要安全機制

尚未完成：一般使用者的端到端操作體驗，以及正式部署驗收

依據：GitHub PR #277–#287；截至 2026-10-12

原本沒有 Excel 排程：人員逐項在 OpenWebUI 向 SRAF 提問

既有 SRAF 能依提示詞查詢資料庫；欠缺的是 Excel 要項驅動、定期執行與集中管理的產情流程。

原本：人工逐項提問

1. 人員自行閱讀 Excel 分析要項
2. 在 OpenWebUI 手動撰寫提示詞
3. 呼叫 SRAF 查詢資料庫資料
4. 閱讀答覆，依需求追問與調整
5. 自行整理結果；無 Excel 排程功能

目標： Excel 驅動自動產情

1. 上傳 Excel ／外部系統送入要項
2. 自動預覽、檢核並保存要項版本
3. 建立單次或週期性的產情任務
4. 系統排程呼叫既有 SRAF 分析流程
5. 查詢執行紀錄並取得 Word 報告

核心改變：從「人看 Excel → 人提問 SRAF」，走向「Excel 設定要項 → 系統自動產情」。

輸入一份要項，形成可持續運作的產情任務

目標端到端流程；不同方塊的開發與驗收狀態，後續逐一說明。



底層共用： SRA 既有匯入、版本管理、權限、排程、執行工作者、檢索分析與報告儲存

**** 目標成果 **** 不必逐項在 OpenWebUI 手動提問，也能保存任務、資料版本、產出與責任歸屬。

長官最關心的：到底丟什麼進去，會拿到什麼？

支援不同的輸入方式；實際欄位由核准的匯入設定檔轉換，不是限制死一張 Excel 格式。

INPUT ｜輸入

資料內容

Excel （.xlsx）、 CSV 、 JSON ； 或直接送入要項 JSON

執行設定

要項名稱、核准的匯入設定版本、單次／定期排程、啟停與權限設定

OUTPUT ｜輸出

- ✓ **產情報告**
  Word / DOCX： 彙整分析結果
- ✓ **任務與執行狀態**
  完成、失敗、執行紀錄
- ✓ **資料與排程版本**
  可辨識每次執行所用要項
- ✓ **操作與稽核證據**
  可追溯誰建立、修改及觸發

例：每週追蹤「特定海域動態」→ 排程蒐整相關資料 → 產出當週 Word 產情報告

不是另做一個系統，而是把原有 SRA 能力整合起來

讓新功能沿用現有資料、角色權限、執行管線與 Word 報告機制。

既有 SRA 管理介面 /editor ＋ 未來的「要項與自動化任務」專區

Product API （新的統一操作入口）

Requirement Import

要項匯入／預覽／ Snapshot

Schedule + Handoff

排程、版本、執行交接

Report Job + AI

Agent / LLM / ES ／ DOCX

共用底座： Auth / RBAC 、 Service Key 、稽核、 SQL / ES 、歷史檔案與既有匯入管理

來源：Product API + Task Center Design v1（2026-10-01）

現況拆成三類：已完成、正在做、下一步

以下是開發完成度，而不是正式上線完成度。

已完成｜合併 main

- Product 資料結構與 ID 管理
- 輸入資料預覽與檢核服務
- 一次性任務建立與初始化
- 週期性任務建立與啟動
- 修改任務／排程提案與核准
- 權限邊界與低階資源保護

進行中｜ PR #287

- 更換既有任務的要項輸入
- Snapshot 版本衝突控制
- 重試與併發一致性收斂
- Codex P1 審查意見修正

後續｜尚未交付

- 對外 Product API ／ Run 觸發
- 整合式要項任務中心 UI
- Service Key 自動化授權驗證
- 真實輸入 E2E 與正式部署
- 瀏覽器操作與安全回歸

核心界線：後端 service 能力 ≠ 對外 API 已完成 ≠ 使用者頁面已完成 ≠ 正式系統已驗收

這幾週不是停在同一件事，而是一段一段完成可驗證的里程碑

時間範圍： 9 月中下旬至 10 月 12 日；日期依 Hosted GitHub 合併紀錄與既有驗證紀錄。

9 月中下旬

既有查詢與技術底座

OpenWebUI 人工提問
技術 Golden Path
尚無 Excel 排程入口

10 / 01–06

整合設計 + 資料基座

PR #277–#279
產品設計、 Schema v4
跨系統 ID 重試保護

10 / 07–08

任務實際建立

PR #280–#284
預覽、單次 / 週期
任務修改與版本草稿

10 / 11–12

權限一致性

PR #285–#286 已合併
PR #287 輸入替換審查中

階段解讀 先證明能執行 → 再做產品控制層 → 最後交付操作介面與正式驗收。

從人工問答變成自動產情，難的不只是讀 Excel，而是整條可靠流程

尤其「排程 + 產情」跨越資料、權限、版本、工作者與外部模型。

① 多階段整合

匯入、資料版本、排程、 AI 分析與 Word 報告，不是同一個操作。

② 斷線／失敗恢復

同一請求重送，不能多生一個任務、多跑一次，或留下半成品。

③ 多人同時操作

更新中的 Excel 與排程版本必須避免相互覆蓋或產生錯配。

④ 安全與可追溯

不同操作權限、稽核軌跡、輸入版本及產出檔案都必須對得起來。

因此採「分階段實作 → 回歸測試 → 程式審查 → 合併」方式，避免一次整包開發造成返工。

表面上看不見的功能，才是自動化能長期運作的關鍵

以已實作的 Product 後端機制為例。

01 重複請求不重做

同一筆建任務／更新任務請求可辨識並恢復，降低重複產出風險。

Idempotency

02 資料版本不搞混

每次執行綁定資料快照及排程版本，避免使用新資料冒充舊執行結果。

Snapshot / Version

03 更新衝突即時阻擋

使用版本比對與交易保護，在同時修改時拒絕不一致的覆寫。

CAS / Transaction

04 不是任何人都能更改

敏感排程啟動權限與資源擁有權設邊界，沿用現有 RBAC 。

Ownership / RBAC

實際作用：減少重複任務、遺漏分析、排程互搶、結果無法追溯等作業風險。

P1.7：讓使用者「更換要項 Excel」時，舊任務仍安全、結果仍一致

PR #287 已開啟，尚未合併；目前 Codex Review 有一項需修正的 P1 finding。

情境：同一個產情任務，兩個人先後更新輸入檔案

原始版本 S0 → A 替換 → S1 → B 替換 → S2

要避免：舊請求回來時蓋掉新版本、重跑產生重複資料、記錄狀態和實際快照對不上。

當前需修正

Codex P1 審查意見

當候選輸入包含錯誤時，版本檢查與失敗紀錄須同一交易完成，避免競爭狀況出現不一致結果。

下一步：修正 → 重跑對應測試 → exact-head Review / CI → 核准後合併

GitHub：PR #287 — input replacement snapshot CAS

「要項任務中心」將整合既有 SRA 管理介面

下圖為規劃中的操作畫面概念示意，並非目前已完成的實際截圖。



規劃功能：上傳與預覽 Excel ｜任務建立｜排程草稿核准｜立即執行｜歷程／DOCX 下載

下一步用「可驗收交付物」推進，不再只以程式模組數量衡量進度

順序以相依關係為主；各階段需完成測試與檢視後才進入下一階段。

近期	01	完成 P1.7	輸入替換、競爭處理、 Review 與合併	完成後端 Product v1 主要控制機制
下一階段	02	Product API	預覽／建立／查詢／修改／觸發／下載	外部系統可透過正式介面送入要項
再下一階段	03	權限 + 任務中心	Role / Service Key 、 SRA UI 、執行紀錄	操作員不再自行串接低階 API
正式交付前	04	E2E 與部署驗收	真實 Excel 、真實 ES/LLM 、 DOCX 、 Browser	有可追溯的整鏈驗證證據

不列入本次 v1： 重做排程引擎、全新登入系統、通用流程編輯器、分散式架構大改。

程式合併 ≠ 正式環境通過：還有幾件事必須驗證

下列是驗收前需管控的客觀風險，應納入下一階段排程。

資料環境	**正式 Elasticsearch 曾有 mapping_incompatible**	校正 mapping / capability 後，重做真實檢索與產情報告驗證
執行穩定	**SQL/ 排程並行、失敗恢復與接手測試未全結案**	正式環境做交易／競爭／失效注入及 worker 驗收
對外可用	**Product API 、介面與瀏覽器 E2E 尚待完成**	以真實 Excel 、權限及 DOCX 回收形成上線驗收案例

管理面建議：先排定測試環境／資料與跨單位配合窗口，避免程式完成後仍卡在正式驗收。

讓產情工作從「一次一次做」，變成「設定一次、持續追蹤」

先建立可測量的效益指標；未完成正式場域量測前，不臆測節省百分比。

01

節省人工作業

減少逐項閱讀 Excel 、重寫提示詞及人工回收結果

02

加快資訊供應

可依設定自動啟動產情，減少人工等待與交接

03

降低作業錯誤

版本綁定、重試安全與權限阻擋降低誤執行

04

提升管理能見度

看得到任務量、成功率、失敗原因、產出歷史

建議追蹤 KPI：人工提問次數、單次產情耗時、按期交付率、成功率、報告可追溯率

一句話結論：後端整合已有具體成果；下一個價值轉折點，是正式 API + 操作介面 + 真實環境通過。

本簡報採用的工程事實，均以目前 Hosted GitHub 為主要依據

主管閱讀時，可將「合併 main」「審查中」「尚待驗收」視為三個不同的狀態。

近期 Product 設計與執行	PR #277–#286：設計、資料結構、任務建立、版本、授權保護。	↗
目前待修與合併	PR #287：Requirement Product input replacement snapshot CAS。	↗
正式設計規格	Product API + Task Center Design v1（2026-10-01）。	↗
分階段計畫	Product Implementation Plan v1：Gate P1–P8。	↗
部署尚待結案事項	Repo .codex/known_issues.md：live ES、SQL、worker、UI E2E。	↗

註： GitHub repository 為 private；文件連結須具相應存取權限。內容反映 2026-10-12 的查詢結果。